Teste Técnico: Assistente de Desenvolvimento, Epavi

Prazo: 72 horas a partir do recebimento
Formato: desafio para fazer em casa, com entrega por escrito (código, documentação e relatório)

---

1. Contexto

A Epavi monitora alarmes de residências, comércios, indústrias e agências bancárias. Todo evento que um painel de alarme envia chega à **central de monitoramento**: disparos, pânico, falta de energia, falha de comunicação, arme, desarme e testes periódicos. Um operador então atende o evento e o encerra com um resultado.

Hoje esses dados ficam presos no sistema da central. A gestão quer **acompanhar a operação em um painel**, sem depender de planilhas montadas à mão.

Sua missão é construir uma **primeira versão funcional** desse sistema, de ponta a ponta:

```
API da central  →  captura  →  banco de dados  →  dashboard
```

O kit vem com uma **API simulada** (`api_mock/`), que faz o papel da central. Leia o `api_mock/README_API.md` para saber como subir a API e quais endpoints ela oferece.

2. Use IA à vontade

O uso de ferramentas de IA (Claude, ChatGPT, Copilot, Cursor etc.) é liberado e incentivado. Na nossa equipe, programar com IA faz parte do dia a dia. O que queremos ver é **como** você usa essas ferramentas: se direciona bem, se revisa o que foi gerado e se entende e consegue explicar o resultado.

No relatório vamos pedir que você explique partes do código com as suas palavras. Ter usado IA não é problema. Não entender o próprio código é.

3. O que você deve construir

3.1 Captura

Um processo que coleta os dados da API e grava no banco. Ele deve:

- autenticar e percorrer todas as páginas;
- **sobreviver às falhas da API** (limite de requisições, erros temporários, lentidão) sem perder dados e sem travar;
- ser **incremental**: na segunda execução em diante, buscar apenas o que mudou, sem baixar tudo de novo;
- poder rodar de forma recorrente (por exemplo, a cada 5 minutos). Explique como você agendaria isso;
- registrar logs do que aconteceu em cada execução.

3.2 Armazenamento

- Use o banco que preferir. **SQLite é perfeitamente aceitável**; PostgreSQL ou outro também servem.
- Modele as tabelas de eventos, clientes e tipos de evento.
- **Garanta a qualidade dos dados.** Os dados vêm de uma integração real e você vai encontrar problemas neles. Identifique, trate e **documente** cada problema que encontrou e a decisão que tomou.
- Rodar a captura duas vezes **não pode** duplicar nem distorcer os dados.

3.3 Dashboard

Um painel web que a gestão da central possa usar. A ferramenta é livre: Streamlit, Dash, Flask/FastAPI com HTML, React, Metabase, Power BI etc. Só precisamos conseguir abrir e usar o painel seguindo o seu README.

Indicadores mínimos:

1. Volume de eventos ao longo do tempo, por dia e por tipo ou categoria.
2. Eventos abertos ou em atendimento **neste momento**.
3. Tempo médio de atendimento, por prioridade e por operador.
4. Ranking dos clientes com mais disparos de alarme, mostrando a taxa de falso alarme.
5. Clientes com problemas técnicos recorrentes (falha de comunicação, energia, bateria).

Filtros mínimos: período, cidade e tipo de cliente.

Diferencial: traga **pelo menos um insight** que a gestão não pediu, mas deveria saber, e mostre esse insight no painel. Há padrões interessantes escondidos nos dados.

O visual conta: queremos um painel que alguém de fora da TI consiga entender sem explicação.

3.4 Documentação

O `README.md` do seu projeto deve ter:

1. **Como rodar**, passo a passo, a partir de uma máquina limpa (dependências, ambiente virtual, comandos).
2. **Arquitetura:** um diagrama simples ou uma descrição do fluxo.
3. **Modelo de dados:** as tabelas e o porquê de cada uma.
4. **Qualidade de dados:** os problemas que você encontrou e como tratou cada um.
5. **Decisões e trade-offs:** o que você escolheu e o que deixou de fora.
6. **Uso de IA:** quais ferramentas usou e em quais etapas. Conte também como você conferiu o que a IA gerou. Se precisou corrigir ou ajustar alguma coisa, dê um exemplo. Se não precisou, explique como validou que estava certo.
7. **Próximos passos:** o que você faria com mais tempo.

4. Entrega

- Envie um **repositório no GitHub** (público, ou privado com acesso liberado) ou um `.zip` com o projeto.

- Inclua **capturas de tela** do dashboard no README.
- **Não inclua** o banco de dados nem ambientes virtuais (`venv` , `node_modules`) no repositório.
- Prazo: **72 horas** após receber este teste. Se não conseguir terminar tudo, entregue o que tiver e explique o que faltou. Isso também é avaliado.

5. Relatório escrito

Junto com o projeto, entregue um arquivo `RELATORIO.md` (ou PDF) de até 2 páginas, respondendo:

1. **Para a gestão (sem jargão técnico):** o que o painel mostra e quais as 3 conclusões mais importantes que você tirou dos dados.
2. Se a captura rodar duas vezes seguidas, o que acontece no banco? Por quê?
3. Se a API ficar fora do ar por 2 horas, o que acontece quando ela voltar?
4. O total de eventos no seu banco é igual ao `total_registros` da API? Se não for, explique a diferença.
5. Escolha um trecho do seu código que trata algum problema dos dados e explique, com as suas palavras, o que ele faz e por que você fez assim.

6. O que vamos observar

Aspecto	O que conta
Funcionamento	Roda seguindo o README? Captura completa, incremental e resiliente?
Dados	Os números estão corretos? Os problemas foram encontrados e tratados?
Código	Organização, clareza, tratamento de erros, facilidade de manter
Dashboard	Responde às perguntas da gestão? Visual claro? Traz insight?
Documentação	Alguém consegue entender e dar continuidade sem você?
Comunicação escrita	Clareza do relatório e do README, para quem é e para quem não é da área técnica
Uso de IA	Usa a IA com critério, revisa e entende o que entrega

Não existe uma única resposta certa. Preferimos **uma solução simples, correta e bem explicada** a uma solução grande e frágil.

Dúvidas sobre o enunciado podem ser enviadas por e-mail ao longo do teste. Saber perguntar também conta.

Boa sorte!